Day 18 — ROC curves, AUC, and decision thresholds

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3, Day 4 — complete line-by-line companion to day18_roc_auc_thresholds.py

Every section below follows the same order: what the idea means and why it matters, then the exact code for it, then a line-by-line walk-through of what each line actually does. Read the explanation before the code — the code will make more sense once you already know what problem it's solving.

0. Imports and setup

Every earlier script (Day 15-17) trained a model and stopped at “accuracy = X”. Day 18 asks a sharper question: that accuracy number came from assuming a passenger with predicted probability > `0.5` is labeled "survived" and everyone else is labeled "died". That 0.5 cutoff was never earned — it's just Python's default. This script exposes that default, measures the model independent of any single cutoff (ROC/AUC), and then picks a cutoff on purpose.

```
import os
import urllib.request

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    roc_curve, roc_auc_score, precision_recall_curve,
    precision_score, recall_score, f1_score, accuracy_score
)
```

→ `os`, `urllib.request` — same as Day 15-17, used only to check whether titanic.csv already exists locally and download it if not

→ `numpy as np` — new this script. Needed for array math when sweeping F1 across every threshold the precision-recall curve produces

→ `roc_curve, roc_auc_score` — the two functions behind the ROC curve and the single AUC number that summarizes it

→ `precision_recall_curve` — like `roc_curve` but plots precision against recall instead of true-positive-rate against false-positive-rate

→ `precision_score, recall_score, f1_score` — compute one number each, at one specific threshold you choose — used in the threshold-sweep table

1. prepare_and_train() — same pipeline, now trains two models

Identical load/clean/encode/split logic from Day 15-17: fill missing Age with the median, drop Cabin, fill missing Embarked with the mode, engineer FamilySize, one-hot encode Sex and Embarked. New in Day 18: it trains both a logistic regression and a random forest in the same function, so the rest of the script can compare their ROC curves side by side.

```
        def prepare_and_train(path=DATA_PATH, url=DATA_URL):
            if not os.path.exists(path):
                urllib.request.urlretrieve(url, path)

            df = pd.read_csv(path)
            df["Age"] = df["Age"].fillna(df["Age"].median())
            df = df.drop(columns=["Cabin"])
            df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
            df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

            y = df["Survived"]
            feature_cols = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
                            "Embarked", "FamilySize"]
            X = pd.get_dummies(df[feature_cols], columns=["Sex", "Embarked"], drop_first=True)

            X_train, X_test, y_train, y_test = train_test_split(
                X, y, test_size=0.2, random_state=42, stratify=y
            )

            logreg = LogisticRegression(max_iter=1000)
            logreg.fit(X_train, y_train)

            forest = RandomForestClassifier(n_estimators=200, max_depth=4, random_state=42)
            forest.fit(X_train, y_train)

            return {
                "X_test": X_test, "y_test": y_test,
                "logreg": logreg, "forest": forest,
            }
```

→ `if not os.path.exists(path)` — only download titanic.csv if it isn't already sitting in the folder; keeps you from re-downloading every run

→ `fillna(df["Age"].median())` — replaces missing ages with the median age, not the mean, because age is right-skewed (a few very old passengers would drag the mean up)

→ `drop(columns=["Cabin"])` — Cabin is missing for most passengers, so it's dropped rather than imputed

→ `pd.get_dummies(..., drop_first=True)` — turns Sex (male/female) into one column {C}Sex_male{E}, and Embarked (S/C/Q) into two columns; {C}drop_first=True{E} avoids redundant columns that all say the same thing

→ `stratify=y` — forces the same survival rate in both train and test splits, so the split itself doesn't accidentally bias the comparison

→ Two separate `.fit()` calls — logistic regression and random forest are trained independently on the exact same X_train/y_train, so any difference in their ROC curves later comes from the model, not from

different data

→ the function `returns` a dictionary instead of a tuple — new pattern this script, used because there are now 4 things to carry forward (X_test, y_test, logreg, forest) and a dict with named keys is easier to read at the call site than `X_test, y_test, logreg, forest = prepare_and_train()`

2. predict_proba() vs predict()

Every classifier in scikit-learn actually computes a probability first, then converts it to a label. `.predict()` only ever shows you the label. `.predict_proba()` shows you the number that label came from — this is the raw material every technique in this lesson is built on.

```
        def show_predict_proba(models):
            logreg = models["logreg"]
            X_test = models["X_test"]

            labels = logreg.predict(X_test)
            probs = logreg.predict_proba(X_test)[:, 1]

            preview = pd.DataFrame({
                "P(survived)": probs[:8].round(3),
                ".predict() label": labels[:8],
                "matches (prob > 0.5)?": (probs[:8] > 0.5) == (labels[:8] == 1),
            })
            return probs
```

→ `logreg.predict_proba(X_test)` returns a 2-column array for every passenger: column 0 is P(died), column 1 is P(survived). The two always add up to 1.0

→ `[:, 1]` — NumPy slicing meaning "every row, only column 1". This keeps just P(survived) and throws away the redundant P(died) column

→ `probs[:8]` — slices the first 8 values just for a readable preview table, not a technical requirement, purely for printing

→ `(probs[:8] > 0.5) == (labels[:8] == 1)` — builds a boolean column that proves `.predict()` is doing nothing more than checking if the probability crosses 0.5. Every value in this column should be True

→ the function returns the full `probs` array (all passengers, not just 8) because every later function in the script needs every probability, not just the preview slice

3. The ROC curve and AUC

What the ROC curve actually plots

Sweep the decision threshold from 1.0 down to 0.0. At every threshold, compute two rates: the true positive rate (TPR = of all passengers who actually survived, what fraction did the model correctly flag?) and the false positive rate (FPR = of all passengers who actually died, what fraction did the model incorrectly flag as survivors?). Plot FPR on the x-axis and TPR on the y-axis as the threshold sweeps — that line is the ROC curve. A model that is just guessing randomly traces the diagonal line

from (0,0) to (1,1). A useful model bulges up and to the left of that diagonal.

What AUC compresses that curve into

AUC ("area under the curve") is a single number between 0 and 1 summarizing the whole ROC curve: the probability that, if you picked one random survivor and one random non-survivor, the model would assign a higher survival probability to the actual survivor. 0.5 means the model's rankings are no better than a coin flip. 1.0 means perfect ranking. Critically, AUC never mentions 0.5 or any other cutoff — it measures how well the model ranks passengers, completely separate from where you eventually draw the line.

```
        def compute_roc(models, logreg_probs):
            y_test = models["y_test"]
            forest_probs = models["forest"].predict_proba(models["X_test"])[:, 1]

            fpr_lr, tpr_lr, _ = roc_curve(y_test, logreg_probs)
            auc_lr = roc_auc_score(y_test, logreg_probs)

            fpr_rf, tpr_rf, _ = roc_curve(y_test, forest_probs)
            auc_rf = roc_auc_score(y_test, forest_probs)

            return {
                "fpr_lr": fpr_lr, "tpr_lr": tpr_lr, "auc_lr": auc_lr,
                "fpr_rf": fpr_rf, "tpr_rf": tpr_rf, "auc_rf": auc_rf,
                "forest_probs": forest_probs,
            }
```

→ `models["forest"].predict_proba(models["X_test"])[:, 1]` — same pattern as step 2, just applied to the random forest instead of logistic regression, so both models can be compared on the same test passengers

→ `roc_curve(y_test, logreg_probs)` takes the true labels and the predicted probabilities — NOT the predicted labels — because it needs to sweep every possible threshold itself, which requires the raw probability, not an already-decided 0/1

→ `fpr_lr, tpr_lr, _ = roc_curve(...)` — the function actually returns 3 arrays (false-positive rates, true-positive rates, and the thresholds that produced them). The underscore _ is Python convention for "I'm receiving this value but I don't need it" — the thresholds array itself isn't used for plotting here

→ `roc_auc_score(y_test, logreg_probs)` — again takes probabilities, not labels, for the same reason: it needs the full ranking, not one fixed cutoff's worth of information

4. The precision/recall tradeoff

Precision answers: "of everyone the model called a survivor, what fraction actually survived?" Recall answers: "of everyone who actually survived, what fraction did the model catch?" These pull in opposite directions. Lower the threshold and the model calls more people survivors — recall goes up (catching more real survivors) but precision drops (more of those calls are wrong). Raise the threshold and the reverse happens. There is no threshold that maximizes both at once; the script proves this by sweeping several thresholds and printing all four metrics side by side.

```
        def show_threshold_tradeoff(models, probs):
            y_test = models["y_test"]
            rows = []
            for t in [0.3, 0.4, 0.5, 0.6, 0.7]:
                preds_at_t = (probs >= t).astype(int)
                rows.append({
                    "threshold": t,
                    "precision": precision_score(y_test, preds_at_t, zero_division=0),
                    "recall": recall_score(y_test, preds_at_t, zero_division=0),
                    "f1": f1_score(y_test, preds_at_t, zero_division=0),
                    "accuracy": accuracy_score(y_test, preds_at_t),
                })
            table = pd.DataFrame(rows)
            return table
```

→ `for t in [0.3, 0.4, 0.5, 0.6, 0.7]` — manually picks 5 candidate thresholds to compare, instead of the one default of 0.5 every earlier day silently used

→ `(probs >= t).astype(int)` — a boolean array (True/False for each passenger) converted to 1/0 integers with `.astype(int)`, because the scoring functions expect integer class labels, not booleans

→ `zero_division=0` — at a high threshold, the model might call zero passengers survivors. Precision would then be 0 divided by 0, which is undefined; this argument tells scikit-learn to report 0 instead of raising an error in that edge case

→ `rows.append({...})` — builds one dictionary per threshold and collects them in a list; `pd.DataFrame(rows)` then turns that list of dicts into one clean table, one row per threshold, in a single line

5. Precision-recall curve and picking an F1-optimal threshold

Rather than guessing 5 thresholds by hand, sweep every threshold the data actually produces and score each one with F1 — the harmonic mean of precision and recall, which only scores high when both are reasonably high at the same time. The threshold with the best F1 is a defensible, data-driven choice instead of an arbitrary guess.

```
        def compute_pr_curve(models, probs):
            y_test = models["y_test"]
            precision, recall, thresholds = precision_recall_curve(y_test, probs)

            denom = precision[:-1] + recall[:-1]
            numer = 2 * precision[:-1] * recall[:-1]
            f1_scores = np.divide(numer, denom, out=np.zeros_like(numer), where=denom > 0)

            best_idx = np.argmax(f1_scores)
            best_threshold = thresholds[best_idx]

            return precision, recall, thresholds, best_threshold
```

→ `precision_recall_curve(y_test, probs)` — like `roc_curve`, sweeps every threshold present in the data and returns precision and recall at each one

→ scikit-learn quietly returns precision and recall arrays that are exactly 1 element longer than the thresholds array (there's a defined precision/recall value at the boundary past the last real threshold, but no threshold number to pair with it) — `[:-1]` trims both arrays down to the same length as thresholds so they line up correctly

→ F1 formula: 2 × precision × recall / (precision + recall) — written out directly as `numer` over `denom` instead of calling `f1_score` in a loop, because this needs to be computed for hundreds of thresholds at once as arrays, not one at a time

→ `np.divide(numer, denom, out=np.zeros_like(numer), where=denom > 0)` — a safe division: at any threshold where both precision and recall are 0, dividing by 0 would normally raise a warning; this tells NumPy to only divide where the denominator is greater than 0, and to fill in 0 everywhere else

→ `np.argmax(f1_scores)` — returns the index position of the single highest F1 score in the whole array, not the score itself. `thresholds[best_idx]` then looks up which actual threshold produced that best score

6. Plotting both curves

Two side-by-side plots make both ideas visible at once: the ROC curve comparing both models' overall ranking ability, and the precision-recall curve marking exactly where the F1-optimal threshold sits along that tradeoff.

```
        def plot_curves(roc_data, pr_data, models, logreg_probs):
            precision, recall, thresholds, best_threshold = pr_data
            fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

            axes[0].plot(roc_data["fpr_lr"], roc_data["tpr_lr"],
                         label=f"Logistic Regression (AUC={roc_data['auc_lr']:.3f})")
            axes[0].plot([0, 1], [0, 1], linestyle="--", color="gray",
                         label="Random guess (AUC=0.5)")

            axes[1].plot(recall, precision, label="Logistic Regression")
            axes[1].scatter(
                [recall[np.argmin(np.abs(thresholds - best_threshold))]],
                [precision[np.argmin(np.abs(thresholds - best_threshold))]],
                label=f"F1-optimal (t={best_threshold:.2f})"
            )

            plt.savefig("day18_roc_pr_curves.png", dpi=150)
```

→ `plt.subplots(1, 2, ...)` — 1 row, 2 columns of plots side by side; `axes[0]` is the left plot (ROC), `axes[1]` is the right plot (precision-recall)

→ `axes[0].plot([0, 1], [0, 1], linestyle="--")` — draws the diagonal reference line from (0,0) to (1,1) representing what a model with zero skill (pure random guessing) would trace

→ the recall/precision arrays from step 5 are already the exact x/y coordinates of the precision-recall curve, so `axes[1].plot(recall, precision)` needs no further computation

→ `np.argmin(np.abs(thresholds - best_threshold))` — finds the index in the thresholds array closest to best_threshold, so the scatter point can be placed at the matching recall/precision coordinate. Needed because best_threshold is a value, not automatically an index into these arrays

7. Deliverable and push

Run the script once with real titanic.csv before pushing. Confirm both `day18_roc_pr_curves.png` was created and the printed AUC / F1-optimal threshold make sense before committing.

```
git add day18_roc_auc_thresholds.py day18_roc_pr_curves.png
git commit -m "Day 18: ROC curves, AUC, and decision thresholds - beyond the default 0.5 c
git push
```